Controlled Packet Capture and 802.11 Protocol Analysis Lab - WPA-PSK Analysis

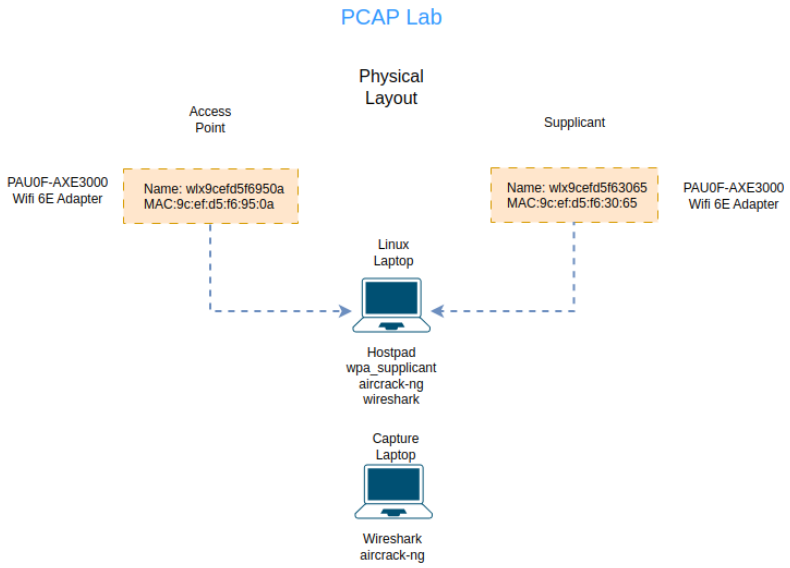
In this post we will explore the authentication process relating to WPA-PSK via packet capture analysis. We will start off with a review of what the frame control field looks like in Wireshark followed by exploration of the MAC frame. We will then conclude with a comparison of a successful and unsuccessful authentication process.

Lab Setup

A picture is worth a thousand words as the saying goes! Following are diagrams depicting the Logical and Physical layout of our Lab environment. Going forward, we will reference the diagrams as required.

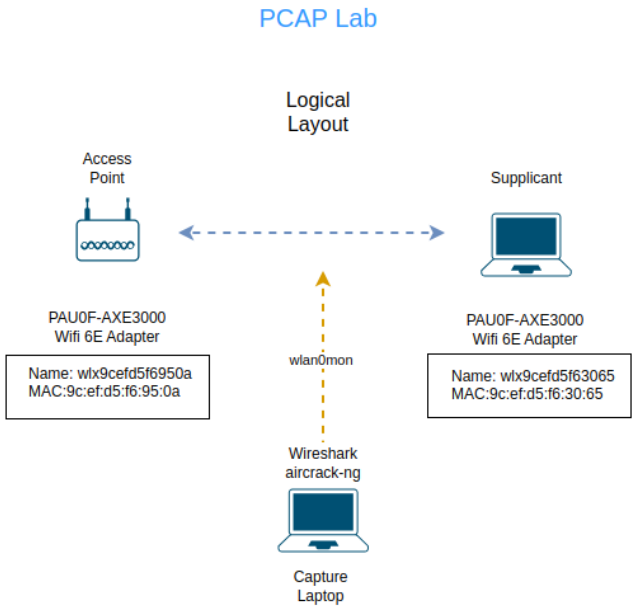
PCAP Lab Physical

Following highlights the physical components of the PCAP Lab. Please refer to the first post in this series for assistance with Lab setup if required.



PCAP Lab Logical

Logically, our lab has one supplicant (Client), one Access Point (AP), and a Laptop acting as a capture point. This laptop will follow our established process of creating a monitor interface via airmon-ng (wlan0mon) for utilization of packet capture. To minimize the length of the post, we will not be going into setup of Wireshark per post as this topic has been established previously. Instead, we will review previously captured traffic as to focus on analysis. Please note, during packet-capture, capture filters were applied to minimize the noise in our pcaps. Using capture filters can assist with minimizing noise in our pcaps, however, when possible, I suggest starting with no capture filters to broaden the capture scope.



Access Point

Let's kick this off by initializing our access point. To do so, we will utilize the following hostapd configuration file. Notice the 'interface' and 'ssid'? Please change the interface to align with your interface naming.

hostapd_wpa2_basic.config

```
interface=wlx9cefd5f6950a
driver=nl80211
ssid=PCAP-Lab
hw_mode=a
channel=36
ieee80211n=1
wmm_enabled=1


# Beacon Interval
beacon_int=100

# Open network for simplest analysis (or add WPA2 below)
auth_algs=1
ignore_broadcast_ssid=0

# Optional WPA2-Personal for handshake capture
wpa=2
wpa_key_mgmt=WPA-PSK
rsn_pairwise=CCMP
wpa_passphrase=Testlab12345
```

Initialize hostapd

```
sudo hostapd -i wlx9cefd5f6950a hostapd_wpa2_basic.config
```

Result

```
wlx9cefd5f6950a: interface state UNINITIALIZED->ENABLED
wlx9cefd5f6950a: AP-ENABLED
```

Client (Supplicant)

Great! Now we have a functioning access-point for our lab. Let's work on the client. We will use the following configuration file when initializing a client connect to the access-point via the wpa_supplicant.

wpa_psk_correct_psk.conf

```
#ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev
update_config=1
country=US

# Optional: randomize source MAC in pre-association probe requests
# preassoc_mac_addr=1

network={
    ssid="PCAP-Lab"
    psk="Testlab12345"
    key_mgmt=WPA-PSK
    proto=RSN
    pairwise=CCMP
    group=CCMP
    ieee80211w=1

    # Send directed probe requests with this SSID
    scan_ssid=1

    priority=1
}
```

Initiate client connection

Prior to initiating a connection attempt to our access-point, we have started Wireshark on the capture laptop, utilizing our monitor interface wlan0mon. Perfect, let's attempt a connection.

```
sudo wpa_supplicant -i wlx9cefd5f63065 -c wpa_psk_correct_psk.conf
```

Client connecting

```
wlx9cefd5f63065: CTRL-EVENT-REGDOM-CHANGE init=USER type=COUNTRY alpha2=US
wlx9cefd5f63065: SME: Trying to authenticate with 9c:ef:d5:f6:95:0a
(SSID='PCAP-Lab' freq=5180 MHz)
wlx9cefd5f63065: Trying to associate with 9c:ef:d5:f6:95:0a (SSID='PCAP-Lab'
freq=5180 MHz)
wlx9cefd5f63065: Associated with 9c:ef:d5:f6:95:0a
wlx9cefd5f63065: CTRL-EVENT-SUBNET-STATUS-UPDATE status=0
wlx9cefd5f63065: WPA: Key negotiation completed with 9c:ef:d5:f6:95:0a
[PTK=CCMP GTK=CCMP]
wlx9cefd5f63065: CTRL-EVENT-CONNECTED - Connection to 9c:ef:d5:f6:95:0a
completed [id=0 id_str=]
```

Great! If we are watching our terminal that we used initialized hostapd, we will see a message indicating success or failure!

hostapd - access-point

```
wlx9cefd5f6950a: interface state UNINITIALIZED->ENABLED
wlx9cefd5f6950a: AP-ENABLED
wlx9cefd5f6950a: STA 9c:ef:d5:f6:30:65 IEEE 802.11: authenticated
wlx9cefd5f6950a: STA 9c:ef:d5:f6:30:65 IEEE 802.11: associated (aid 1)
wlx9cefd5f6950a: AP-STA-CONNECTED 9c:ef:d5:f6:30:65
wlx9cefd5f6950a: STA 9c:ef:d5:f6:30:65 RADIUS: starting accounting session
4B29D1103141494F
wlx9cefd5f6950a: STA 9c:ef:d5:f6:30:65 WPA: pairwise key handshake completed
(RSN)
wlx9cefd5f6950a: EAPOL-4WAY-HS-COMPLETED 9c:ef:d5:f6:30:65
wlx9cefd5f6950a: AP-STA-DISCONNECTED 9c:ef:d5:f6:30:65
```

Success! We see that our client successfully connected to our access-point utilizing WPA-PSK and the proper credentials. Let's see what this looks like via our packet capture!

Analysis

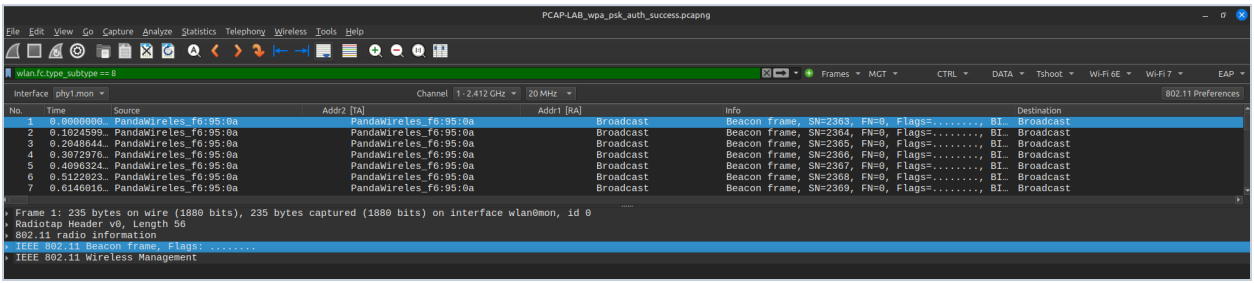
Prior to diving in WPA-PSK analysis, let's have a quick look at a Management Frame (Beacon). Beacons and Probe Response frame are packed with valuable information. It is important to align how the Frame Control field and the MAC frame appear in packet captures. I feel having this visual will greatly assist with understanding, not only the individual fields, but how the full frame is constructed.

Beacon

Let's start by applying a wireshark filter for our Management Frame (Beacon). If you downloaded and imported the recommended Wireshark profiles, you can also select, MGT, BEACON. Doing so will apply a proper beacon display filter.

Wireshark Display Filter

```
wlan.fc.type_subtype == 8
```



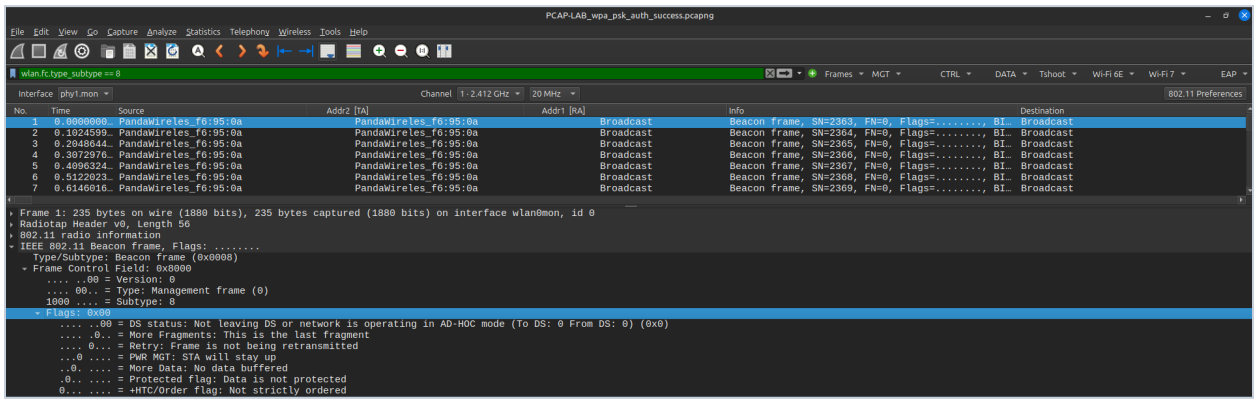
Frame Control

Looking at our Beacon frame, we can identify the 'Frame Control' field. Pause and take notice to how the frame control fields align with our previous discussion relating to FC!

We see the FC Ver, Type, Subtype (8)(Indicating beacon), to DS/from DS/,More Fragments, Retry, Power Management, More Data, Protected, +HTC/Order.

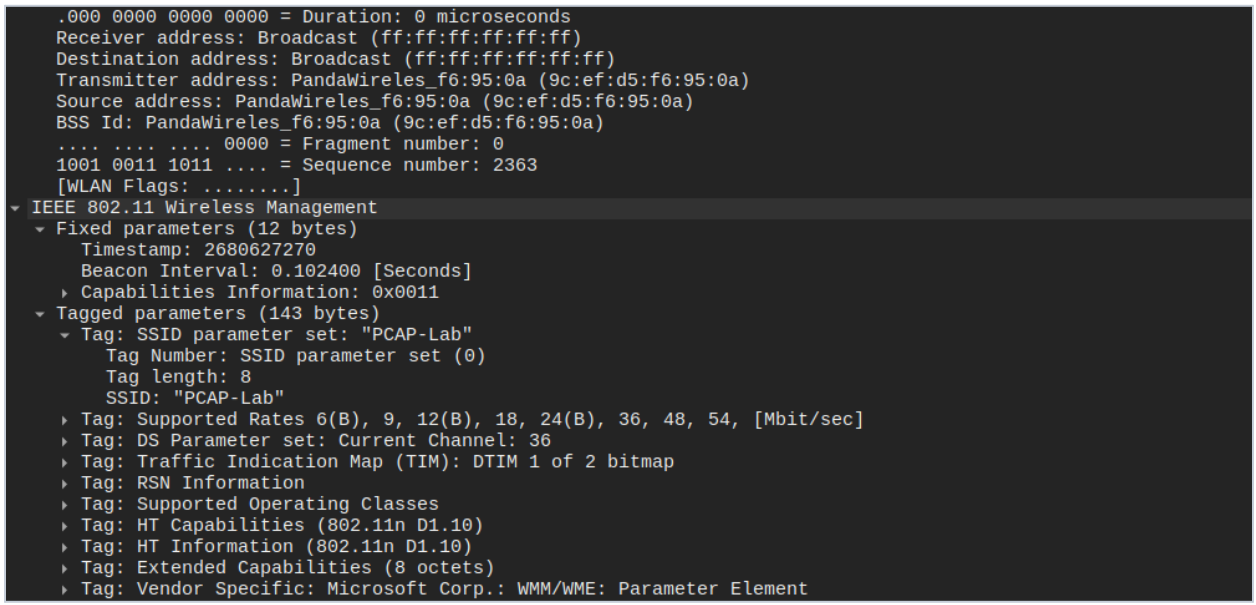
Recall the purpose of the Frame Control field! To set up context for that Frame. Some important notes.

- DS Field - to/from DS - Setup context or flow of frame. "Leaving DS or going to DS"
- Retry - Tells us whether the frame is a retransmission. Some retries are normal, but excessive retries are a Red Flag to start looking for root cause. Break out the spectrum analyzer!
- Protected - Tells us whether the frame body is encrypted. You'll see it set on encrypted data frames, and on some management frames (like Deauthentication) when PMF is in use.



Frame Body

Tagged parameters are known as Information Elements (IEs). Each IE follows the same format consisting of an Element ID, a Length, and the information itself. As new 802.11 amendments are introduced, new IEs are added to advertise their capabilities, such as HT (802.11n), VHT (802.11ac), HE (802.11ax), and EHT (802.11be). Other IEs, such as SSID, Supported Rates, and RSN (Robust Security Network), describe the network itself. Take note of the RSN element in our beacon as it advertises WPA-PSK as the AKM (Authentication and Key Management) and CCMP as the cipher.



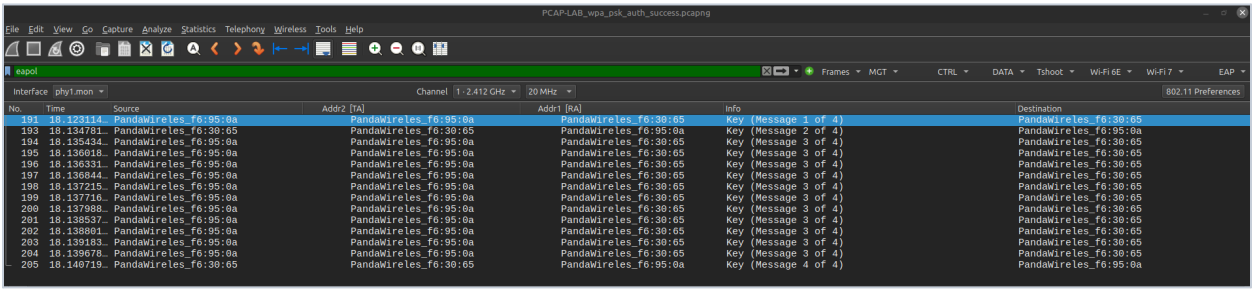
WPA-PSK Analysis

Now that we established a visual of not only the Frame Control field and MAC field, also what and where we can find them via a packet capture. Note, we used a beacon frame for simplicity and for the abundance of information.

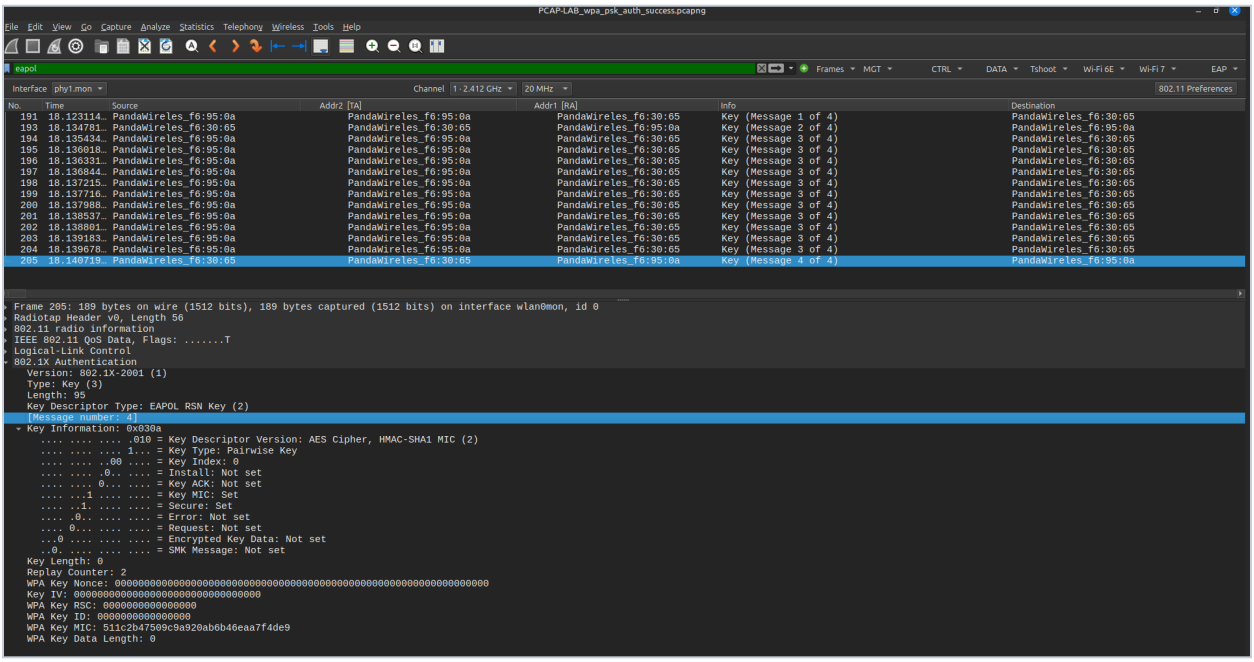
Back to our WPA-PSK capture.

WPA-PSK Successful Authentication

The conversation from the supplicant to the access-point will utilize EAPOL. To that end, let's apply a Wireshark display filter of "EAPOL"



Immediately, we can see 4 messages indicating the 4-WAY Handshake was successful.



So what is actually happening in each of those messages? Here's the short version:

- Message 1 (AP to Client) - The AP kicks things off by sending its random number, the ANonce.
- Message 2 (Client to AP) - The client answers with its own random number (the SNonce) and a MIC. This is the first proof that the client actually knows the PSK.
- Message 3 (AP to Client) - The AP confirms with its own MIC and hands over the group key (GTK).

- Message 4 (Client to AP) - The client confirms the keys are installed, and encrypted traffic can start flowing.

WPA-PSK Failed Authentication

Now that we have seen a successful 4-Way handshake, let's have a look at a failed authentication request. For this test we will use the following wpa_supplicant configuration file.

wpa_psk_incorrect_psk.conf

Notice we changed the 'PSK' to an incorrect password.

```
#ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev
update_config=1
country=US

# Optional: randomize source MAC in pre-association probe requests
# preassoc_mac_addr=1

network={
    ssid="PCAP-Lab"
    psk="WrongPSK"
    key_mgmt=WPA-PSK
    proto=RSN
    pairwise=CCMP
    group=CCMP
    ieee80211w=1

    # Send directed probe requests with this SSID
    scan_ssid=1

    priority=1
}
```

Initiate client connection

```
sudo wpa_supplicant -i wlx9cefd5f63065 -c wpa_psk_incorrect_psk.conf
```

Client connecting

Notice Handshake failed!!

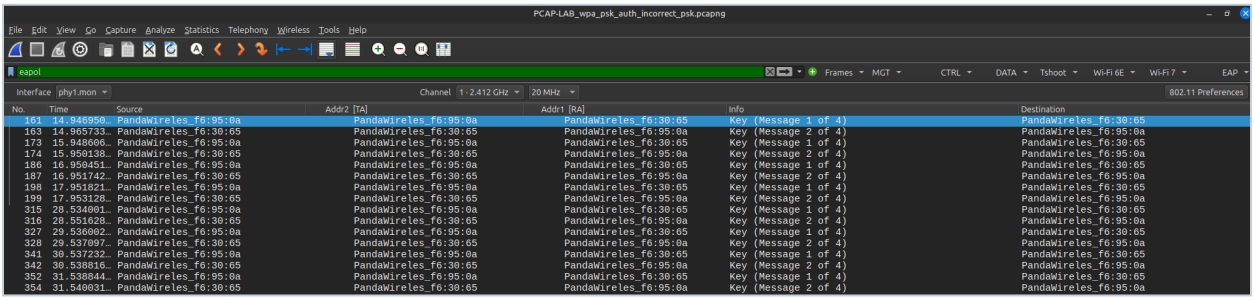
```
wlx9cefd5f63065: SME: Trying to authenticate with 9c:ef:d5:f6:95:0a
(SSID='PCAP-Lab' freq=5180 MHz)
wlx9cefd5f63065: Trying to associate with 9c:ef:d5:f6:95:0a (SSID='PCAP-Lab'
freq=5180 MHz)
wlx9cefd5f63065: Associated with 9c:ef:d5:f6:95:0a
wlx9cefd5f63065: CTRL-EVENT-SUBNET-STATUS-UPDATE status=0
wlx9cefd5f63065: CTRL-EVENT-DISCONNECTED bssid=9c:ef:d5:f6:95:0a reason=15
wlx9cefd5f63065: WPA: 4-Way Handshake failed - pre-shared key may be
incorrect
wlx9cefd5f63065: CTRL-EVENT-SSID-TEMP-DISABLED id=0 ssid="PCAP-Lab"
auth_failures=1 duration=10 reason=WRONG_KEY
^Cwlx9cefd5f63065: CTRL-EVENT-DSCP-POLICY clear_all
wlx9cefd5f63065: CTRL-EVENT-DSCP-POLICY clear_all
nl80211: deinit ifname=wlx9cefd5f63065 disabled_11b_rates=0
wlx9cefd5f63065: CTRL-EVENT-TERMINATING
```

hostapd - access-point

```
wlx9cefd5f6950a: STA 9c:ef:d5:f6:30:65 IEEE 802.11: authenticated
wlx9cefd5f6950a: STA 9c:ef:d5:f6:30:65 IEEE 802.11: associated (aid 1)
wlx9cefd5f6950a: AP-STA-POSSIBLE-PSK-MISMATCH 9c:ef:d5:f6:30:65
wlx9cefd5f6950a: AP-STA-POSSIBLE-PSK-MISMATCH 9c:ef:d5:f6:30:65
wlx9cefd5f6950a: AP-STA-POSSIBLE-PSK-MISMATCH 9c:ef:d5:f6:30:65
wlx9cefd5f6950a: AP-STA-POSSIBLE-PSK-MISMATCH 9c:ef:d5:f6:30:65
wlx9cefd5f6950a: STA 9c:ef:d5:f6:30:65 IEEE 802.11: deauthenticated due to
local deauth request
```

Wireshark

Immediately we can see only messages 1 and 2 of the handshake exchanged!



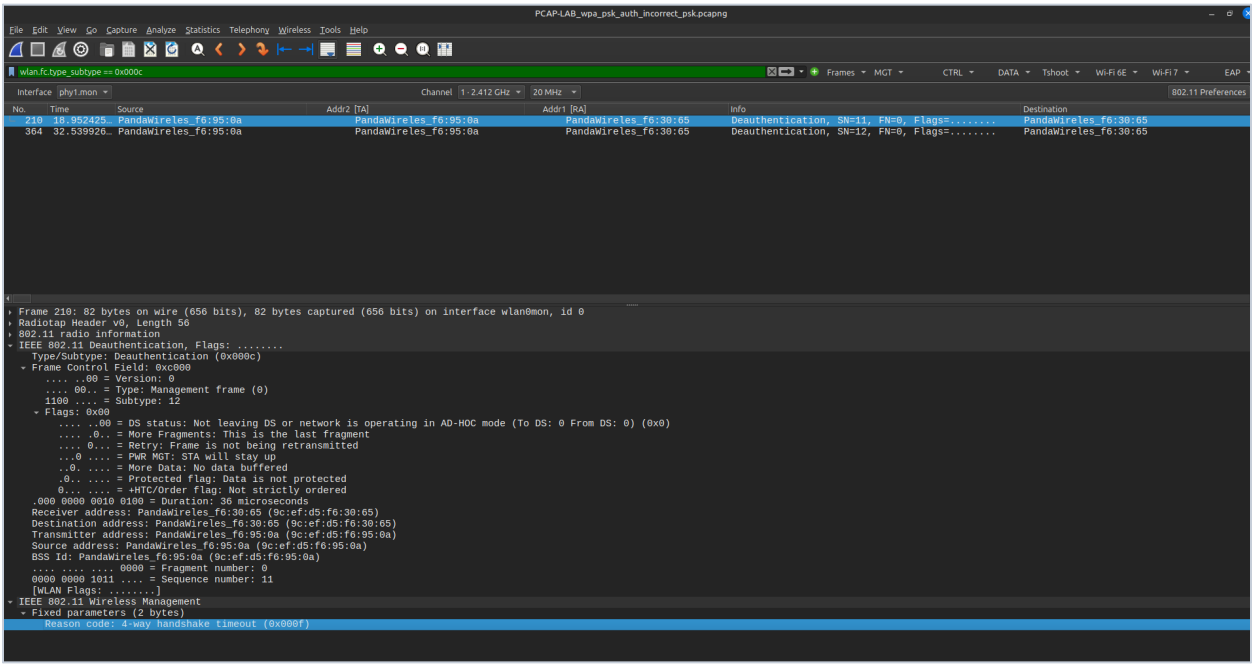
Notice that we only ever see messages 1 and 2, repeated over and over. Here's what's going on. The client built its keys from the wrong PSK, so the MIC it sends in message 2 doesn't check out on the AP's side. Without a valid MIC, the AP won't move on to message 3. Instead, it resends message 1 and tries again until it gives up. So if you see messages 1 and 2 repeating with no message 3, think PSK mismatch.

Let's have a look closer!

Now we see messages 1 and 2 only! Ok, but how do we see the reason! Well, the reason code can be found in Deauthentication or Disassociation frames! Let's look at the Deauthentication frame!

```
wlan.fc.type_subtype == 0x000c
```

Recall, Deauthentication is a Management Frame! Also take note to the Subtype and Reason Code!



In our capture, the Reason Code is 15 (0x000f), which means 4-Way Handshake timeout. The AP waited for a message 2 it could validate, never got one, and ended the conversation. If you want to jump straight to these in a busy capture, filter on the reason code:

```
wlan.fixed.reason_code == 15
```

Quick Triage Tip

If you only see message 1 over and over, with no message 2 coming back, don't blame the PSK just yet. The client isn't answering at all, so start by looking at RF, the client itself, or whether your capture is simply missing frames.

Conclusion

In this post we have explored how we can utilize our PCAP Lab to host an access-point offering WPA-PSK AKM. We then explored how to initiate a client connection utilizing wpa_supplicant. Once initiated, we captured the client and access-point exchange via a monitoring laptop. We then took a look at the packet capture, identifying the frame control and MAC frame as it relates to Wireshark views. Next we dove into analysis of WPA-PSK exchange highlighting a successful 4-way handshake against an unsuccessful handshake. In our next post, we will take a look at how to identify an 802.1X exchange and how we can identify the EAP type in use.